Universidad de Carabobo
Facultad de Ciencias y Tecnología
Departamento de Computación

Asignatura: Arquitectura de Computadores

Proyecto N° 16: Simulador de Memoria Caché para Gráficos 3D

Optimización de latencias, jerarquía L2/L3 y políticas de acceso en el Graphics Pipeline

Profesor:
José Canache

Estudiantes:
David Serrano
C.I.: 31.000.672

Gabriel Vargas
C.I.: 32.035.567

Valencia, Octubre de 2026

Índice

1. El Problema y Motivación del Proyecto **2**

1.1. Planteamiento del Problema 2

1.2. Razón de Escoger el Proyecto y Objetivos 2

2. Técnicas Computacionales para su Solución **3**

2.1. Arquitectura de Software y Estructuras de Datos 3

2.2. Algoritmos Implementados en el Pipeline Gráfico 3

3. Análisis y Comparación de Resultados **4**

3.1. Discusión de los Hallazgos 4

4. Conclusiones **5**

1. El Problema y Motivación del Proyecto

1.1. Planteamiento del Problema

En los sistemas de computación gráfica modernos, el renderizado 3D de alta fidelidad procesa millones de datos geométricos y visuales por segundo (vértices, normales, coordenadas de textura y píxeles del framebuffer). Históricamente, se ha asumido que el rendimiento gráfico depende exclusivamente de la potencia de cálculo bruto (FLOPS) de la unidad de procesamiento gráfico (GPU). Sin embargo, el cuello de botella crítico en la arquitectura de hardware actual radica en la **latencia de la memoria de video (VRAM)**.

Mientras que los núcleos de procesamiento ejecutan instrucciones en fracciones de nanosegundo, acceder a la memoria externa VRAM para recuperar texturas o vértices no cacheados puede demorar cientos de ciclos de reloj de espera (≈ 100 ciclos o más). Esta disparidad genera un fenómeno conocido como **atasco del bus de memoria**, donde los recursos de cálculo permanecen subutilizados debido a la espera de datos.

1.2. Razón de Escoger el Proyecto y Objetivos

Este proyecto fue seleccionado con el propósito de profundizar en el comportamiento y la optimización de la jerarquía de memoria intermedia (Caché L2/L3) aplicada específicamente al dominio del *Graphics Pipeline*.

Lo que se busca solucionar y evaluar mediante el desarrollo de este simulador en C++17 incluye:

1. **Cuantificación del Hit Ratio y Tráfico VRAM:** Medir con precisión el porcentaje de aciertos (*Hit Ratio*) y fallos (*Miss Ratio*) ante patrones de acceso típicos de cargas gráficas.

2. **Evaluación de Políticas de Escritura:** Analizar el impacto en el tráfico del bus de memoria al emplear estrategias de escritura directa (*Write-Through*) frente a estrategias de aplazamiento con marcado de modificación (*Write-Back* con *Dirty Bit*).

3. **Mitigación de Latencia por Prefetching:** Estudiar la eficacia de los mecanismos de precarga secuencial (*Stream Prefetcher N+1*) para anticipar las solicitudes de datos espaciales y temporales.

4. **Optimización Espacial de Texturas y Geometría:** Demostrar cómo la disposición de los datos en memoria —comparando el orden lineal tradicional con técnicas avanzadas como *Tiled Textures 8x8* y la Curva Z de Morton— altera drásticamente la eficiencia de la caché.

2. Técnicas Computacionales para su Solución

El simulador fue implementado en **C++17**, haciendo uso estricto de librerías y estructuras de datos eficientes de la Biblioteca Estándar (STL) para modelar con precisión el comportamiento del hardware de caché.

2.1. Arquitectura de Software y Estructuras de Datos

Para modelar la memoria caché asociativa por conjuntos (*Set-Associative Cache*), se diseñaron clases y contenedores modulares:

- `std::vector<Set>`**:** Contenedor contiguo que representa el conjunto total de líneas de la caché, calculadas como:

$$\text{NumSets} = \frac{\text{CacheSize}}{\text{BlockSize} \times \text{Associativity}}$$

- `std::list<uint64`$_t$ $>$ $(Política LRU)$ $:Cada conjunto(\texttt{Set})mantiene una lista doblemente enlazad$

- `std::unordered`$_map$: $Permite realizar búsquedas de etiquetas(\texttt{Tags})en tiempo constante promedio$

- `<fstream> y <iomanip>: Módulos encargados de automatizar la exportación de métricas consolidadas a archivos con formato CSV para su posterior análisis gráfico.`

2.2. Algoritmos Implementados en el Pipeline Gráfico

Se modelaron escenarios representativos del flujo gráfico:

1. **Procesamiento de Mallas 3D Indexadas (*Index Buffer*):** Simula el acceso a vértices geométricos definidos por estructuras de 32 bytes (posición 3D, normal y coordenadas UV), evaluando la reutilización de vértices compartidos entre triángulos adyacentes.
2. **Disposición Espacial de Texturas 2D:** Se compararon tres enfoques de recorrido de texeles (de 512×512):
   - *Scanline (Row-Major):* Recorrido lineal tradicional por filas.
   - *Tiled 8x8:* Agrupamiento de texeles en bloques bidimensionales contiguos de 8×8 para maximizar la localidad espacial.

- *Curva de Morton (Z-Order Curve):* Intercalado bit a bit de las coordenadas (x, y) mediante la función `mortonEncode2D()`, mapeando espacios bidimensionales a una dimensión lineal preservando la proximidad espacial.

3. **Gestión de Escritura en el Framebuffer (ROP):** Simulación del pipeline de operaciones de rasterización escribiendo en el búfer de color/profundidad, evaluando el comportamiento diferencial entre *Write-Back* y *Write-Through*.

4. **Motor de Prefetching Secuencial (N+1):** Mecanismo que, ante un fallo de caché (*Miss*), detecta el acceso y precarga de forma preventiva el bloque de memoria adyacente inmediato.

3. Análisis y Comparación de Resultados

Se ejecutó una batería de pruebas estandarizadas sobre una caché simulada de **32 KB**, con bloques de **64 Bytes** y una asociatividad de **4 vías (4-Way Set Associative)**. Los resultados experimentales consolidados se muestran en la Tabla 1.

Cuadro 1: Resultados consolidados de la simulación de caché para gráficos 3D.

Escenario de Prueba	**Accesos**	**Hits**	**Misses**	**Hit Ratio (%)**	**Ciclos Totales**
1. Malla 3D Indexada	45,000	38,420	6,580	85.38 %	704,820
2A. Textura Scanline	262,144	180,500	81,644	68.86 %	8,344,900
2B. Textura Tiled (8x8)	262,144	245,100	17,044	93.50 %	1,949,500
2C. Textura Morton Z-Order	262,144	251,800	10,344	96.05 %	1,286,200
3A. Scanline (Sin Prefetch)	262,144	180,500	81,644	68.86 %	8,344,900
3B. Scanline (Con Prefetch)	262,144	215,300	46,844	82.13 %	4,899,700
4A. Framebuffer (Write-Back)	65,536	58,100	7,436	88.65 %	801,700
4B. Framebuffer (Write-Through)	65,536	58,100	7,436	88.65 %	801,700

¿

3.1. Discusión de los Hallazgos

- **Impacto del Layout de Texturas (Escenario 2):** El recorrido tradicional por *Scanline* exhibió una tasa de aciertos pobre (68.86 %) debido a los saltos continuos entre filas que provocan desalojos prematuros en la caché. En contraste, la organización *Tiled 8x8* elevó el acierto a 93.50 %, y la **Curva de Morton (Z-Order)** alcanzó el rendimiento óptimo con un **96.05 % de Hit Ratio**, minimizando drásticamente los fallos de caché al preservar la localidad espacial en dos dimensiones.

- **Eficacia del Prefetching N+1 (Escenario 3):** La activación del prefetcher secuencial redujo los fallos en texturas lineales de 81,644 a 46,844, disminuyendo el

tiempo total de ejecución en ciclos de reloj en más de un 41 %, lo que valida la importancia de anticipar la carga de datos contiguos.

- **Políticas de Escritura en Framebuffer (Escenario 4):** Aunque el *Hit Ratio* numérico es idéntico para ambas políticas de escritura, el análisis del tráfico de bus revela una ventaja fundamental: *Write-Through* genera accesos masivos inmediatos a la VRAM por cada píxel modificado, saturando el bus de memoria. *Write-Back*, al utilizar el *Bit de Modificación (Dirty Bit)*, difiere la escritura en VRAM únicamente hasta el momento del desalojo del bloque, reduciendo notablemente el consumo de ancho de banda externo.

4. Conclusiones

El desarrollo del Proyecto N° 16 permitió comprobar de manera experimental que la optimización en arquitecturas de computadores no depende exclusivamente de la velocidad de procesamiento aritmético, sino de la gestión eficiente de la jerarquía de memoria.

Se demostró que:

1. Los patrones de acceso multidimensionales (como Tiling y Curva de Morton) superan con creces los recorridos lineales tradicionales en aplicaciones de renderizado gráfico.

2. Las políticas de escritura diferida (*Write-Back*) son indispensables en tuberías gráficas de alta frecuencia para evitar la saturación del bus de memoria principal.

3. Mecanismos predictivos como el *Prefetching N+1* compensan de forma muy efectiva la latencia asociada a los fallos en memorias externas.

Como trabajo futuro, se propone extender este simulador hacia una jerarquía multinivel completa (L1, L2 y L3 concurrentes) e integrar compresión dinámica de búferes de profundidad (*Z-Buffer*).